Baze de date II

CURS 4: Cursori; TABLE si VARRAY

Baza de date utilizata. Tabela STUD

MATR	NUME	AN	GRUPA	DATAN	LOC	TUTOR	PUNCTAJ	CODS
1456	GEORGE	4	1141A	12-MAR-82	BUCURESTI		2890	11
1325	VASILE	2	1122A	05-OCT-84	PITESTI	1456	390	11
1645	MARIA	3	1131B	17-JUN-83	PLOIESTI		1400	11
3145	ION	1	2112B	24-JAN-85	PLOIESTI	3251	1670	21
2146	STANCA	4	2141A	15-MAY-82	BUCURESTI		620	21
3251	ALEX	5	2153B	07-NOV-81	BRASOV		1570	21
2215	ELENA	2	2122A	29-AUG-84	BUCURESTI	2146	890	21
4311	ADRIAN	3	2431A	31-JUL-83	BUCURESTI		450	24
3514	FLOREA	5	2452B	03-FEB-81	BRASOV		3230	24
1925	OANA	2	2421A	20-DEC-84	BUCURESTI	4311	760	24
2101	MARIUS	1	2412B	02-SEP-85	PITESTI	3514	310	24
4705	VOICU	2	2421B	19-APR-84	BRASOV	4311	1290	24

Tabelele SPEC si BURSA

CODS	NUME	DOMENIU
11	MATEMATICA	STIINTE EXACTE
21	GEOGRAFIE	UMANIST
24	ISTORIE	UMANIST

TIP	PMIN	PMAX	SUMA
FARA BURSA	0	399	
BURSA SOCIALA	400	899	100
BURSA DE STUDIU	900	1799	150
BURSA DE MERIT	1800	2499	200
BURSA DE EXCEPTIE	2500	9999	300

Ce sunt cursorii

- Pentru execuţia cererilor SQL prezente în blocurile PL/SQL şi pentru stocarea rezultatelor acestora (cum e cazul cererilor SELECT) Oracle foloseşte un spaţiu de lucru accesibil utilizatorului prin intermediul unui obiect numit cursor.
- Acesta poate fi de exemplu folosit pentru a parcurge linie cu linie rezultatul şi pentru a afla câte linii are acesta.

Tipuri de cursori

- Cursorii sunt de două tipuri:

1. **Cursori impliciţi**. Ei sunt definiţi automat de Oracle pentru orice cerere de tip DML, inclusiv pentru cererile SELECT .. INTO care returnează o singură linie prezentate în capitolele anterioare.
   - Aceşti cursori nu au un nume declarat de utilizator iar cererea SQL asociată lor este ultima cerere SQL executată.
2. **Cursori expliciţi**. Aceştia sunt definiţi de utilizator care precizează numele cursorului şi cererea SQL asociată.

Utilizare

- Pentru a folosi cursori expliciţi paşii sunt următorii:

1. În zona de declaraţii se defineşte cursorul (nume, cerere SQL asociată).
2. În zona executabilă (dupa BEGIN, deci inclusiv în zona de tratare a excepţiilor) se pot folosi instrucţiunile de lucru cu un cursor: OPEN, FETCH şi CLOSE.
3. Pentru orice tip de cursor se pot folosi metodele asociate acestora: %ISOPEN, %NOTFOUND, %FOUND şi %ROWCOUNT

Observaţii

- Instrucţiunile OPEN, FETCH şi CLOSE sunt specifice cursorilor expliciţi şi nu pot fi folosite pentru cei impliciţi (pentru aceştia operaţiile respective sunt executate automat de Oracle).
- Pentru cursorii impliciţi valorile atributelor acestora (%NOTFOUND, %FOUND şi %ROWCOUNT) se pot afla folosind un nume de cursor generic - *SQL* - care desemnează cel mai recent cursor implicit deschis.

Cursori impliciti

- Ultima cerere executată a unui bloc PL/SQL are asociat un cursor cu numele generic SQL.
- Trebuie remarcat că spre deosebire de cererile SELECT . . INTO, celelalte cereri DML nu generează excepţii în cazul în care nici o linie nu este afectată (de exemplu în cazul DELETE sau UPDATE).

Cursori impliciti

- Atributele cursorului SQL vor avea următoarele valori:
- SQL%ROWCOUNT: numărul de linii afectate de cerere
- SQL%FOUND: TRUE dacă există linii afectate de cerere şi FALSE altfel
- SQL%NOTFOUND: TRUE dacă nu există linii afectate de cerere şi FALSE altfel
- SQL%ISOPEN: nu se aplică cursorilor impliciţi.

Exemplu

```
BEGIN
UPDATE STUD
SET PUNCTAJ = PUNCTAJ + 100
WHERE CODS = 21;
dbms_output.put_line('Au fost modificate'
  || SQL%ROWCOUNT || ' linii');
IF sql%found THEN
  dbms_output.put_line('Found TRUE');
ELSE
  dbms_output.put_line('Found FALSE');
END IF;
```

Exemplu

```
IF sql%notfound THEN
  dbms_output.put_line('NotFound TRUE');
ELSE
  dbms_output.put_line('NotFound FALSE');
END IF;
END;
```

- Rezultat:

```
Au fost modificate 4 linii
Found TRUE
NotFound FALSE
```

Cursori expliciti

- Aşa cum am menţionat paşii de lucru sunt:

1. Declararea cursorului
2. Deschiderea cursorului
3. Încărcarea liniei curente (este mereu o cerere SELECT!)

Declarare

- Sintaxa declaraţiei unui cursor este:

```
DECLARE
  CURSOR nume_cursor IS
   cerere_select;
```

- unde:
  - **nume_cursor** este un identificator PL/SQL care denumeşte cursorul.
  - **cerere_select** este o cerere SELECT care nu trebuie să conţină clauza INTO. Variabila/lista de variabile în care se încarcă linia curentă din rezultat va fi specificată în instrucţiunea FETCH.

Deschidere

- Prin execuţia instrucţiunii OPEN se 'deschide' un cursor.
- Efectul practic este următorul:
  - se execută cererea SQL asociată cursorului.
  - se aduce în zona de memorie a acestuia rezultatul cererii (o tabelă rezultat).
  - se setează înregistrarea (linia) curentă la prima linie.

Deschidere

- Sintaxa instrucţiunii este:

```
OPEN nume_cursor;
```

Observaţie:

- Dacă cererea SQL nu întoarce nici o linie sau întoarce mai multe linii **nu** se ridică excepţiile NO_DATA_FOUND respectiv TOO_MANY_ROWS!

Incarcare linie

- Rezultatul lui OPEN este o tabelă rezultat iar indicatorul liniei curente este setat la prima.
- Prin instrucţiunea FETCH:
  - Se încarcă linia curentă într-o mulţime de variabile sau o singură variabilă înregistrare (având mai multe câmpuri). Corespondenţa este poziţională
  - Se incrementează indicatorul liniei curente.
  - Dacă nu s-a putut încărca o linie (mai sunt linii în rezultat) %FOUND întoarce FALSE.

Incarcare linie

- Sintaxa instrucţiunii este:

```
FETCH nume_cursor INTO
  lista_variabile |
  variabila_inregistrare
```

FETCH lista

- În cazul în care încărcarea se face într-o listă de variabile trebuie ca:
  - numărul de variabile din listă să fie acelaşi cu numărul de coloane din rezultatul cererii SELECT
  - tipurile variabilelor din listă să fie aceleaşi cu tipurile coloanelor din rezultatul cererii SELECT

FETCH inregistrare

- În cazul în care încărcarea se face într-o variabilă înregistrare trebuie ca:
  - numărul de câmpuri al variabilei să fie acelaşi cu numărul de coloane din rezultatul cererii SELECT
  - tipurile câmpurilor variabilei să fie aceleaşi cu tipurile coloanelor din rezultatul cererii SELECT
- Se poate defini o variabilă de acelaşi tip cu liniile rezultatului folosind construcţia **nume_cursor%rowtype**.

Atribute cursor

- Atributele cursorului sunt ca niste functii care pot fi consultate după ce s-a făcut primul FETCH şi ele au următoarele valori:

1. **`nume_cursor%ROWCOUNT`**: numărul de linii încărcate cu FETCH de la deschiderea cursorului.
2. **`nume_cursor%FOUND`**: TRUE dacă ultimul FETCH a încărcat o nouă linie şi FALSE altfel.

Atribute cursor

- Atributele cursorului ...

3. **`nume_cursor%NOTFOUND`**: TRUE dacă ultimul FETCH nu a încărcat o nouă linie şi FALSE altfel.
4. **`nume_cursor%ISOPEN`**: TRUE dacă acel cursor e deschis şi FALSE altfel

- Notă: In tabelul urmator expresia 'ultimul FETCH' denumeşte execuţia care nu mai întoarce o linie de rezultat.

Valori atribute

Folosit		%FOUND	%ISOPEN	%NOTFOUND	%ROWCOUNT
înainte de	OPEN	excepție	FALSE	excepție	excepție
după	OPEN	NULL	TRUE	NULL	0
înainte de	Primul FETCH	NULL	TRUE	NULL	0
după	Primul FETCH	TRUE	TRUE	FALSE	1
înainte de	Următoarele FETCH	TRUE	TRUE	FALSE	1
după	Următoarele FETCH	TRUE	TRUE	FALSE	depinde de date
înainte de	Ultimul FETCH	TRUE	TRUE	FALSE	depinde de date
după	Ultimul FETCH	FALSE	TRUE	TRUE	depinde de date
înainte de	CLOSE	FALSE	TRUE	TRUE	depinde de date
după	CLOSE	excepție	FALSE	excepție	excepție

Exemplul 1

- Listarea numelor studenţilor de la specializarea cu cod 21.
- Parcurgerea rezultatului se face cu un ciclu LOOP şi ieşirea este determinată de atributul %NOTFOUND:

```
DECLARE
  -- declarare cursor
 CURSOR studenti IS SELECT * FROM STUD WHERE
  CODS = 21;
  -- variabila pentru incarcare linie
 v_stud  studenti%rowtype;
BEGIN
-- deschidere cursor
OPEN studenti;
```

Exemplul 1

```
LOOP
  -- incarcarea liniei curente
  FETCH studenti INTO v_stud;
  -- iesire cand nu mai sunt linii
  EXIT WHEN studenti%notfound;
  dbms_output.put_line(studenti%rowcount||
                       ' Student:
  '||v_stud.nume);
END LOOP;
-- inchidere cursor
CLOSE studenti;
END;
```

Exemplul 1

- Rezultatul execuţiei este:

```
1 Student: ION
2 Student: STANCA
3 Student: ALEX
4 Student: ELENA
```

Exemplul 2

- Exemplul 2: Rezultate returnate de ISOPEN, FOUND

```
DECLARE
 CURSOR studenti IS SELECT * FROM STUD WHERE
  CODS = 21;
 v_stud studenti%rowtype;
BEGIN
-- inca nu e deschis
IF studenti%isopen THEN
  dbms_output.put_line('Cursorul e deschis');
ELSE
  dbms_output.put_line('Cursorul nu e
  deschis');
END IF;
```

Exemplul 2

```
-- se deschide cursorul
OPEN studenti;
IF studenti%isopen THEN
  dbms_output.put_line('Cursorul e deschis');
ELSE
  dbms_output.put_line('Cursorul nu e deschis');
END IF;
-- este deschis, se incarca o linie
FETCH studenti INTO v_stud;
IF studenti%found THEN
  dbms_output.put_line('Exista rezultat');
ELSE
  dbms_output.put_line('Nici macar o linie');
END IF;
```

Exemplul 2

```
-- inchidere cursor
CLOSE studenti;
IF studenti%isopen THEN
   dbms_output.put_line('Cursorul
  e deschis');
ELSE
   dbms_output.put_line('Cursorul
  nu e deschis');
END IF;
END;
```

Exemplul 2

- Rezultatul execuţiei blocului de mai sus este:

```
Cursorul nu e deschis
Cursorul e deschis
Exista rezultat
Cursorul nu e deschis
```

FOR pentru cursori

- Pentru a uşura parcurgerea liniilor tabelei rezultat asociată unui cursor deschis în PL/SQL există un ciclu FOR special.
- Sintaxa lui este următoarea:

```
FOR v_inregistrare IN cursor
  LOOP
   instructiuni;
END LOOP
```

FOR pentru cursori

- Observaţii:
  - Variabila înregistrare nu trebuie definită de utilizator, acest lucru făcându-se automat.
  - Domeniul de valabilitate al acestei variabile este corpul ciclului.
  - OPEN, FETCH şi CLOSE se execută automat.
  - Se execută câte un pas al ciclului pentru fiecare linie a rezultatului.
  - Ieşirea din ciclu se face automat la terminarea liniilor din rezultat.

Exemplu

```
DECLARE
  CURSOR studenti IS
    SELECT * FROM STUD WHERE CODS = 21;
BEGIN
FOR v_stud IN studenti LOOP
  -- ciclul deschide cursorul,
  -- v_stud nu e declarata
  -- nu e necesar FETCH
  dbms_output.put_line(studenti%rowcount||
        ' Student: '||v_stud.nume);
END LOOP;  -- inchide automat cursorul
END;
```

FOR cu cursori impliciti

- Acest tip de ciclu FOR se poate folosi şi pentru **cursori impliciţi**, ca în exemplul de mai jos:

```
BEGIN
-- nu e necesara declararea cursorului si
-- nici a variabilei v_stud
FOR v_stud IN (select * from stud where cods=21)
   LOOP
   -- nu e necesar FETCH
   dbms_output.put_line('Student: '
         ||v_stud.nume);
END LOOP;
-- nu e necesara inchiderea cursorului
END;
```

- În acest caz returnarea a 0 sau mai multor linii nu duce la ridicarea de excepţii!

Cursori cu parametri

- Unui cursor i se pot transmite parametri. În acest caz:
- La declarare se specifică parametrii formali şi tipul lor (fără să se specifice dimensiunea, deci VARCHAR2 şi nu VARCHAR2(10) de exemplu). Aceşti parametri sunt folosiţi în cererea SQL asociată.
- La deschidere se specifică parametrii actuali.
- Sintaxa declarării unui parametru este:

```
parametru [IN] tipdedate [{:= |
  DEFAULT} expresie]
```

Exemplu

```
DECLARE
  CURSOR studenti (v_cods number :=21,
                   v_loc varchar2:='BUCURESTI') IS
  SELECT nume FROM stud
  WHERE cods = v_cods and loc = v_loc;
  i number;
BEGIN
FOR i IN 1..5 LOOP -- cod specializare = i*5+1
 dbms_output.put_line('Specializarea ' || (i*5+1));
 FOR v_stud in studenti((i*5+1), 'PLOIESTI') LOOP
    dbms_output.put_line(studenti%rowcount||
                         ' Student: '||v_stud.nume);
 END LOOP;  -- inchide automat cursorul
END LOOP; -- ciclul FOR i in 1..5
```

Exemplu

```
dbms_output.put_line('Cu parametrii impliciti');
FOR v_stud IN studenti LOOP
   dbms_output.put_line(studenti%rowcount||
   'Student: ' ||v_stud.nume);
END LOOP;  -- inchide automat cursorul
END;
```

Exemplu

- Rezultatul este:

```
Specializarea 6
Specializarea 11
1 Student: MARIA
Specializarea 16
Specializarea 21
1 Student: ION
Specializarea 26
Cu parametrii impliciti
1 Student: STANCA
2 Student: ELENA
```

Cursori cu subcereri

- În clauza FROM a cererii SQL care defineşte un cursor poate să apară o subcerere SQL.
- Ea se evaluează o singură dată şi rezultatul său este folosit pentru calculul rezultatului cererii globale.
- Exemplu: declaraţia cursorului conţine pe clauza FROM o subcerere SQL care returnează o tabelă cu trei coloane conţinând numărul şi numele specializării precum şi numărul de studenţi pentru specializările cu mai mult de 3 studenţi:

Cursori cu subcereri

```
DECLARE
CURSOR s IS SELECT t1.cods, t1.nume, cati
FROM SPEC t1, (SELECT cods, count(*) cati
               FROM stud GROUP BY cods) t2
WHERE t1.cods = t2.cods AND cati > 3;
BEGIN
FOR spc IN s LOOP
  dbms_output.put_line(s%rowcount||
    ' Specializarea: '|| spc.cods||
  ' '||spc.nume|| ' ' ||spc.cati);
END LOOP;
END;
```

Cursori cu subcereri

- Rezultatul va fi:

```
1 Specializarea: 21 GEOGRAFIE 4
2 Specializarea: 24 ISTORIE 5
```

- Bineînţeles subcererile pot fi prezente şi pe clauza WHERE, fie corelate, fie necorelate.

Referinte catre cursori

- Aceste referinţe (de tip REF CURSOR) sunt similare pointerilor din limbajele C sau Pascal.
- Definirea se face cu:

```
TYPE nume_tip_pointer IS REF CURSOR
                [RETURN tip_inregistrare];
v_pointer_cursor nume_tip_pointer;
```

- Clauza RETURN se foloseşte în definirea pachetelor (package) pentru a specifica tipul de date întors de cursor.
- Deschiderea cursorului se face în acest caz cu:
- OPEN v_pointer_cursor FOR cerere_sql;
- Deci asocierea cererii SQL se face nu la declarare ci la deschiderea cursorului.

Exemplu

```
DECLARE
TYPE Referinta IS REF CURSOR RETURN stud%rowtype;
c_ref Referinta;  -- variabila de tip REF CURSOR
s stud%rowtype;   -- variabila pentru incarcarea
                  -- unei linii
BEGIN
-- deschidere cursor cu specificarea cererii
OPEN c_ref FOR select * from stud where cods=11;
LOOP
  FETCH c_ref INTO s;
  EXIT WHEN c_ref%notfound;
  dbms_output.put_line(c_ref%rowcount||
     ' Nume: '|| s.nume || ' Punctaj '|| s.punctaj);
END LOOP;
CLOSE c_ref;
```

Exemplu

```
-- deschidere cursor cu alta cerere SQL
OPEN c_ref FOR select * from stud where
  loc='BRASOV';
LOOP
  FETCH c_ref INTO s;
  EXIT WHEN c_ref%notfound;
  dbms_output.put_line(c_ref%rowcount||
     ' Nume: '|| s.nume|| ' Localitate '||
  s.loc);
END LOOP;
END;
```

Exemplu

- Rezultatul va fi:

```
1 Nume: GEORGE Punctaj 2890
2 Nume: VASILE Punctaj 390
3 Nume: MARIA Punctaj 1400
1 Nume: ALEX Localitate BRASOV
2 Nume: FLOREA Localitate BRASOV
3 Nume: VOICU Localitate BRASOV
```

Restrictii REF CURSOR

- Printre restricţiile privind variabilele REF CURSOR menţionăm:
  - Nu se pot declara în pachete (package)
  - Nu se pot compara pentru egalitate, inegalitate sau valoare nulă
  - Nu li se poate asigna o valoare nulă

Restrictii REF CURSOR

- Restrictii… :
  - Tipul REF CURSOR nu poate fi folosit în SQL la definirea coloanelor unei tabele sau vederi.
  - Nu poate fi tipul elementelor unei colecţii
  - Nu sunt interschimbabile cu variabilele tip cursor: de exemplu nu le putem folosi în cicluri FOR pentru cursori (mai sus am folosit LOOP).

Excepţia ROWTYPE_MISMATCH

- Când se încearcă încărcarea unei linii într-o variabilă care nu corespunde ca tip se ridică excepţia ROWTYPE_MISMATCH.
- În exemplul de mai jos se încearcă încărcarea unei linii cu mai multe valori
- într-o variabilă scalară. În acest caz se va ridica excepţia menţionată:

```
DECLARE
TYPE Referinta IS REF CURSOR;
c_ref Referinta;  -- variabila de tip REF CURSOR
s varchar2(10);   -- variabila pentru incarcarea
                  -- unei linii. NU E CORECTA!
```

Excepţia ROWTYPE_MISMATCH

```
BEGIN
-- deschidere cursor cu specificarea cererii
OPEN c_ref FOR select * from stud where
  cods=11;
LOOP
  FETCH c_ref INTO s;   -- aici exceptie!
  EXIT WHEN c_ref%notfound;
END LOOP;
CLOSE c_ref;
EXCEPTION
  WHEN rowtype_mismatch THEN
    dbms_output.put_line('Exceptie
  rowtype_mismatch');
END;
```

SELECT . . FOR UPDATE

- În cazul în care prin intermediul unui cursor se execută operaţii de actualizare a tabelelor se poate folosi clauza FOR UPDATE pentru a bloca liniile necesare în momentul în care se deschide cursorul
- Implicit Oracle blochează o linie în momentul în care aceasta e actualizată.
- De asemenea, când se foloseşte CURRENT OF pentru a modifica liniile unei tabele acestea trebuie blocate înainte cu FOR UPDATE.

SELECT . . FOR UPDATE

- Sintaxa este:

```
DECLARE CURSOR nume_cursor [(parametri)] IS
   SELECT . . .
   . . .
   FOR UPDATE [OF coloana] [NOWAIT]
```

SELECT . . FOR UPDATE

- În momentul deschiderii cursorului liniile găsite de cererea SELECT sunt blocate.
- Ele sunt deblocate la sfârşitul tranzacţiei (COMMIT). După ce modificările au fost comise nu se mai pot încărca linii dintr-un cursor definit cu FOR UPDATE.
- NOWAIT specifică faptul că dacă liniile respective sunt blocate în acel moment Oracle să nu aştepte (aceasta fiind acţiunea standard).
- Exemplu: liniile tabelei STUD sunt blocate pentru ca pe parcursul actualizării să nu fie modificate de alţi utilizatori:

Exemplu

```
DECLARE
CURSOR c_stud IS select nume, loc, rowid
                 from stud FOR UPDATE NOWAIT;
v_nume stud.nume%type; v_loc stud.loc%type;
v_rowid rowid;
v_nr number;
BEGIN
OPEN c_stud; v_nr := 0;
LOOP
  FETCH c_stud into v_nume, v_loc, v_rowid;
  EXIT WHEN c_stud%notfound;
  dbms_output.put_line(v_nume||' '|| v_loc);
```

Exemplu

```
UPDATE stud SET punctaj = punctaj * 1.1
  WHERE rowid = v_rowid;
  -- aici e recomandat CURRENT OF c_stud
  v_nr := v_nr + 1;
END LOOP;
dbms_output.put_line('Actualizate: '||v_nr);
CLOSE c_stud;
COMMIT;
EXCEPTION
  WHEN others THEN
    dbms_output.put_line(SQLCODE|| ' ' ||
      SQLERRM);
END;
```

Exemplu

1. În cazul în care există linii blocate de alţi utilizatori se va genera o excepţie (din cauza lui NOWAIT).

- Mesajul va fi:

```
-54 ORA-00054: resource busy and acquire with
  NOWAIT specified
```

2. Dacă liniile nu sunt blocare de alt utilizator rezultatul este:

```
GEORGE BUCURESTI
VASILE PITESTI
. . . . . .
MARIUS PITESTI
VOICU BRASOV
Actualizate: 12
```

SELECT . . FOR UPDATE

- De asemenea prin clauza FOR UPDATE se blochează linii dintr-o tabelă aflată pe clauza FROM a unei cereri SQL multi-tabelă.
- Pentru aceasta trebuie specificată coloana care se doreşte modificată:

```
FOR UPDATE OF coloana
```

Exemplu

```
DECLARE
CURSOR c_stud IS
  select st.nume, st.rowid, sp.nume
  from stud st,spec sp
  where st.cods=sp.cods and
        sp.nume='MATEMATICA'
  FOR UPDATE of punctaj NOWAIT;
v_stnume stud.nume%type;
v_spnume spec.nume%type;
v_rowid rowid; v_nr number;
```

Exemplu

```
BEGIN
OPEN c_stud;
v_nr := 0;
LOOP
  FETCH c_stud INTO v_stnume, v_rowid, v_spnume;
  EXIT WHEN c_stud%notfound;
  dbms_output.put_line(v_stnume||' '||v_spnume);
  UPDATE stud SET punctaj = punctaj * 1.1
  WHERE rowid = v_rowid;
  -- aici e recomandat CURRENT OF c_stud
    v_nr := v_nr + 1;
END LOOP;
dbms_output.put_line('Actualizate: '||v_nr);
```

Exemplu

```
CLOSE c_stud;
COMMIT;
EXCEPTION
   WHEN others THEN
      dbms_output.put_line(SQLCODE||'
   '||SQLERRM);
END;
```

- Rezultat:

```
GEORGE MATEMATICA
VASILE MATEMATICA
MARIA MATEMATICA
Actualizate: 3
```

WHERE CURRENT OF

- În cazul în care folosim FOR UPDATE linia curentă a unui cursor se poate identifica şi prin construcţia folosită în WHERE :

```
CURRENT OF nume_cursor
```

- Exemplu:

```
DECLARE
CURSOR c_stud IS select nume, loc
                 from stud FOR UPDATE;
v_nume stud.nume%type; v_loc
  stud.loc%type;
v_nr number;
```

Exemplu

```
BEGIN
OPEN c_stud; v_nr := 0;
LOOP
   FETCH c_stud INTO v_nume, v_loc;
   EXIT WHEN c_stud%notfound;
   dbms_output.put_line(v_nume||'
   '||v_loc);
   UPDATE stud1
   SET punctaj = punctaj * 1.1
   WHERE CURRENT OF c_stud;  -- folosire
   v_nr := v_nr + 1;
END LOOP;
```

Exemplu

```
COMMIT;
dbms_output.put_line('Actualizate: '
  ||v_nr);
CLOSE c_stud;
EXCEPTION
  WHEN others THEN
    dbms_output.put_line(SQLCODE||
     ' ' ||SQLERRM);
END;
```

Exemplu

- Rezultatul va fi:

```
GEORGE BUCURESTI
VASILE PITESTI
MARIA PLOIESTI
ION PLOIESTI
STANCA BUCURESTI
ALEX BRASOV
ELENA BUCURESTI
ADRIAN BUCURESTI
FLOREA BRASOV
OANA BUCURESTI
MARIUS PITESTI
VOICU BRASOV
Actualizate: 12
```

TIPURI COMPUSE: TABLE ŞI VARRAY

Asemanari

- TABLE si VARRAY (ambele se mai numesc si colectii) se aseamănă între ele în următoarele privinţe:

1. O colecţie de acest fel poate fi stocată într-o celulă a unei tabele din baza de date (deci o tabelă poate avea o coloană de tip colecţie TABLE sau VARRAY). În cazul tablourilor asociative (TABLE .. INDEX BY) acest lucru nu era posibil.
2. Accesul la elemente se face prin indicele acestora care porneşte de la 1.
3. În ambele cazuri variabilele de acest tip pot fi iniţializate la declarare şi li se pot atribui valori în porţiunea executabilă folosind constructori de aceeaşi formă.

Asemanari

- TABLE si VARRAY se aseamănă...

4. Numărul de elemente dintr-o astfel de colecţie poate varia pe parcursul execuţiei programului (dar în cazul VARRAY nu poate depăşi limita maximă declarată)
5. Elementele unei astfel de colecţii sunt toate de acelaşi tip de date.
6. Limitările privind tipul elementelor sunt aceleaşi în cele două cazuri.

Deosebiri

1. Numărul maxim de elemente ale tipului TABLE nu este fixat la descrierea lui, spre deosebire de VARRAY unde se specifică această limită.
2. Indicii elementelor unui VARRAY se păstrează prin stocarea în baza de date şi regăsirea lui pe când în cazul TABLE acest lucru nu este adevărat.

- Din acest motiv tipul TABLE seamănă cu mulţimile din limbajele de programare Pascal şi C spre deosebire de tipul VARRAY care este asemănător cu tablourile.

Deosebiri

3. În cazul unui VARRAY indicii elementelor sunt succesivi, pornind de la indicele 1, crescător cu o unitate.
- La TABLE deşi intiţial indicii elementelor sunt succesivi, prin ştergerea unor elemente pot să apară 'goluri' (operaţia de ştergere 'din mijloc' nu este posibilă la VARRAY).

3. Ca stocare fizică în baza de date, datele de tip VARRAY se memorează fie în interiorul tabelei (pentru dimensiune mai mică de 4 KB) fie în spaţiul asociat tabelei (tablespace).
4. În cazul TABLE stocarea valorilor se face într-o aşa numită 'tabelă de stocare' (store table) asociată tabelei de bază.

Limitari

- În cazul declarării lor în PL/SQL tipul elementelor poate fi oricare tip valid PL/SQL cu excepţia REF CURSOR
- În cazul folosirii lor în SQL, pe lângă REF CURSOR sunt interzise şi tipurile:
  - BINARY_INTEGER, PLS_INTEGER
  - BOOLEAN
  - LONG, LONG RAW
  - NATURAL, NATURALN
  - POSITIVE, POSITIVEN
  - REF CURSOR
  - SIGNTYPE
  - STRING

Declarare

- Sintaxa definiţiei acestor tipuri este următoarea:
- TABLE:

```
TYPE nume_tip IS TABLE OF tip_element [NOT
  NULL];
```

- VARRAY:

```
TYPE nume_tip IS {VARRAY | VARYING ARRAY}
(indice_maxim) OF tip_element [NOT NULL];
```

unde:

- **nume_tip** - Numele tipului definit de acea declaraţie
- **indice_maxim** - Dimensiunea maximă pentru acel VARRAY
- **tip_element** - tipul elementelor colecţiei
- **NOT NULL** - specifică opţional că elementele nu pot fi nule

Exemple

- TABLE: Declararea a două colecţii care pot conţine un număr nedefinit de denumiri de cursuri la care sunt înscrişi doi studenţi:

```
TYPE ListaCursuri IS TABLE OF
  VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
```

Exemple

- VARRAY: Declararea a două colecţii care pot conţine maxim 25 de denumiri de cursuri la care sunt înscrişi doi studenţi:

```
TYPE ListaCursuri IS VARRAY(25)
  OF VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
```

- Se observă că se poate folosi construcţia %TYPE pentru a defini o variabilă colecţie ca având acelaşi tip cu altă variabilă colecţie.

%TYPE şi %ROWTYPE

- Pentru tipul elementelor se pot folosi de asemenea construcţiile %TYPE şi %ROWTYPE:

```
TYPE NumeStud IS TABLE OF stud.nume%TYPE;
TYPE ListaStud IS TABLE OF stud%ROWTYPE;
CURSOR c_spec IS SELECT * FROM spec;
TYPE ListaSpec IS VARRAY(20) OF
  c_spec%ROWTYPE;
```

- În primul caz, NumeStud este tipul unor colecţii de valori scalare de acelaşi tip cu coloana NUME din tabela STUD.
- În celelalte două cazuri ListaStud şi ListaSpec sunt colecţii având ca elemente înregistrări definite pe baza structurii tabelei stud respectiv a cursorului c_spec.

Elemente de tip înregistrare

- Elementele unor colecţii de acest tip pot fi şi înregistrări definite de utilizator:

```
DECLARE
TYPE Student IS RECORD (
  nume VARCHAR2(40),
  varsta DATE);
TYPE grupa IS VARRAY(25) OF Student;
TYPE seria IS TABLE OF Student;
```

Iniţializarea la declarare

- Iniţializarea unor colecţii de acest tip se poate face la declarare prin construcţia:

```
:= nume_tip_colectie(lista_de_valori)
```

- Exemple:

```
TYPE ListaCursuri IS TABLE OF
  VARCHAR2(30);
ListaMea ListaCursuri :=
  ListaCursuri('Analiza',
          'Algebra', 'Fizica', 'Limba
  engleza');
```

Iniţializarea la declarare

- sau similar în cazul VARRAY:

```
TYPE ListaCursuri IS VARRAY(25)
  OF VARCHAR2(30);
ListaMea ListaCursuri :=
  ListaCursuri('Analiza',
            'Algebra', 'Fizica',
  'Limba engleza');
```

Iniţializarea la declarare

- În cazul în care constructorul a fost apelat fără valori între paranteze se generează o colecţie vidă dar nenulă:

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri :=
  ListaCursuri();
BEGIN
   IF ListaMea IS NOT NULL THEN
     .   .   .  -- condiția de mai sus
  returneaza TRUE!
```

Iniţializarea la declarare

- În schimb dacă o colecţie nu este iniţializată testul de NOT NULL returnează FALSE (deci IS NULL returnează TRUE):

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
   IF ListaMea IS NOT NULL THEN
     .   .   .  -- condiția de mai sus
  returneaza FALSE!
```

Atribuirea

- Se poate lăsa neiniţializată colecţia la declarare şi iniţializa în zona executabilă. Exemplul următor este doar pentru TABLE dar similar se procedează pentru VARRAY:
- Exemplu:

```
TYPE ListaCursuri IS TABLE OF
  VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza',
        'Algebra', 'Fizica', 'Limba
  engleza');
```

Atribuirea

- Lista de valori poate conţine valori nule:

```
TYPE ListaCursuri IS VARRAY(25) OF
  VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL,
  NULL,
         'Algebra', NULL, 'Fizica', 'Limba
  engleza');
```

- În ultimele două exemple au fost asignate colecţii conţinând 4 respectiv 7 elemente (au fost numărate şi valorile NULL).

Atribuiri intre colectii

- Două colecţii de acelaşi tip se pot atribui una alteia. Dacă însă tipul diferă rezultă o excepţie.

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
TYPE AltTip IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
ListaLui ListaMea%Type;
ListaEi  AltTip;
BEGIN
  ListaLui := ListaCursuri('Analiza', NULL, NULL,
         'Algebra', NULL, 'Fizica', 'Limba engleza');
  ListaMea := ListaLui;
  ListaEi := ListaMea; -- va genera exceptie
  dbms_output.put_line('Numar de elemente: '||
   ListaMea.count);
END;
```

Compararea colecţiilor

- O colecţie poate fi testată dacă este nulă sau nu cu IS NULL şi IS NOT NULL.
- În schimb nu putem folosi = sau <> pentru a testa egalitatea sau inegalitatea a două colecţii.
- Din acest motiv o coloană care este de tip colecţie nu poate apărea în cereri SQL în conjuncţie cu clauzele DISTINCT, GROUP BY sau ORDER BY.

Metode pentru TABLE şi VARRAY

- În manipularea datelor de tip TABLE şi VARRAY putem folosi metodele menţionate anterior: EXISTS, COUNT, FIRST, LAST, PRIOR, NEXT şi DELETE.
- Metoda DELETE nu se poate aplica la VARRAY. Se foloseşte TRIM pentru a şterge elemente de la sfârşitul tabloului.

Metode pentru TABLE şi VARRAY

- În plus se pot folosi:

`LIMIT`	**Doar pentru VARRAY: numărul maxim de elemente permis. Pentru celelalte tipuri întoarce NULL.**
`EXTEND[(n[,i)]]`	**Extinde tabloul cu un element nul, *n* elemente nule sau n elemente egale cu elementul cu indice *i*.**
`TRIM[(n)]`	**Şterge un element, respectiv *n* elemente de la sfârşitul tabloului.**

Exemplu: Table

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL, 'Algebra', NULL,
    'Fizica', 'Limba engleza');
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 7
  ListaMea.Extend(3, 2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 10
  ListaMea.Delete(7);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 9
  ListaMea.Delete(3, 6);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 5
  ListaMea.Trim(2);
  dbms_output.put_line('Numar de elemente: '|| ListaMea.count); -- 3
END;
```

Exemplu: VARRAY

```
DECLARE
TYPE ListaCursuri IS VARRAY(25) OF VARCHAR2(30);
ListaMea ListaCursuri;
BEGIN
  ListaMea := ListaCursuri('Analiza', NULL, NULL,
         'Algebra', NULL, 'Fizica', 'Limba engleza');
  dbms_output.put_line('Numar de elemente: '||
   ListaMea.count); -- 7
  ListaMea.Extend(3, 2);
  dbms_output.put_line('Numar de elemente: '||
   ListaMea.count); -- 10
  ListaMea.Trim(2);
  dbms_output.put_line('Numar de elemente: '||
 ListaMea.count); -- 3
END;
```

Exemple

- Metodele FIRST, LAST, NEXT şi PRIOR ne permit parcurgerea unei variabile de tip TABLE sau VARRAY.
- În unele exemple de mai jos am folosit colecţii TABLE cu elemente şterse.

Exemple

- Exemplul 1: Parcurgerea unei colecţii cu ciclu FOR. Acest bloc ridică excepţie dacă există elemente şterse.

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
i NUMBER;
BEGIN
  ListaMea := ListaCursuri('Analiza',
                           'Algebra', 'Sport',
 'Fizica', 'Limba engleza', 'Programare');
  FOR i IN ListaMea.FIRST .. ListaMea.LAST LOOP
    dbms_output.put_line('Curs: '||i||
                         ' '||ListaMea(i));
  END LOOP;
END;
```

Exemple

- Exemplul 1: Parcurgerea unei colecţii cu ciclu FOR. Acest bloc ridică excepţie dacă există elemente şterse.

```
Curs: 1 Analiza
Curs: 2 Algebra
Curs: 3 Sport
Curs: 4 Fizica
Curs: 5 Limba engleza
Curs: 6 Programare
```

Exemple

- Exemplul 2: Parcurgere folosind LOOP pentru a evita excepţia menţionată anterior:

```
DECLARE
TYPE ListaCursuri IS TABLE OF VARCHAR2(30);
ListaMea ListaCursuri;
i NUMBER;
BEGIN
  ListaMea := ListaCursuri('Analiza', 'Algebra', 'Sport',
   'Fizica', 'Limba engleza', 'Programare');
  ListaMea.Delete(3,4);
  i := ListaMea.FIRST;
  LOOP
    dbms_output.put_line('Curs: '||i||
                         ' '||ListaMea(i));
    i := ListaMea.NEXT(i);
    EXIT WHEN i IS NULL;
  END LOOP;
END;
```

Exemple

- Rezultat:

```
Curs: 1 Analiza
Curs: 2 Algebra
Curs: 5 Limba engleza
Curs: 6 Programare
```

Continuarea in cursul urmator

Sfârşitul cursului 4